ANHANGUERA

ENGENHARIA DE SOFTWARE

GABRIEL DAS CHAGAS SILVA

Classificação Iris com TensorFlow

Linguagem de Programação

Unidade 4 - Aplicações com Python | Aula 4 - Machine Learning

2026

GABRIEL DAS CHAGAS SILVA

Classificação Iris com TensorFlow

Relatório de aula prática apresentado à Anhanguera, no curso de Engenharia de Software, para a disciplina Linguagem de Programação.

Aluno: Gabriel das Chagas Silva
Disciplina: Linguagem de Programação
Unidade 4 - Aplicações com Python | Aula 4 - Machine Learning
Data: 13 de outubro de 2026

Ambiente de execução

Os códigos e testes desta entrega foram executados localmente em Python. Os notebooks .ipynb incluídos são compatíveis com o Google Colab e contêm as saídas reais da execução. As capturas mostram o notebook executado localmente; não representam uma sessão no serviço Colab.

Dados e procedimento

O conjunto Iris, carregado pelo scikit-learn, possui 150 amostras, quatro medidas em centímetros e três espécies: setosa, versicolor e virginica. Pandas permite examinar a estrutura. Foram separados 30 exemplos para teste e os outros 120 foram divididos em 96 para treino e 24 para validação, preservando as proporções das classes com stratify.

StandardScaler foi ajustado somente nas 96 amostras de treino. Sua média e desvio foram utilizados para transformar validação, teste e novas flores. Esse procedimento evita que informações do teste influenciem o pré-processamento. Trata-se de padronização, uma técnica de escalonamento, e não de transformar todos os valores para o intervalo de 0 a 1.

A rede TensorFlow/Keras tem quatro entradas, duas camadas ocultas de 16 e 8 neurônios com ReLU e uma saída de três neurônios com softmax. A perda sparse_categorical_crossentropy é apropriada para rótulos inteiros. Adam otimiza os pesos, e EarlyStopping monitora val_loss com paciência 15 e restaura os melhores pesos da validação.

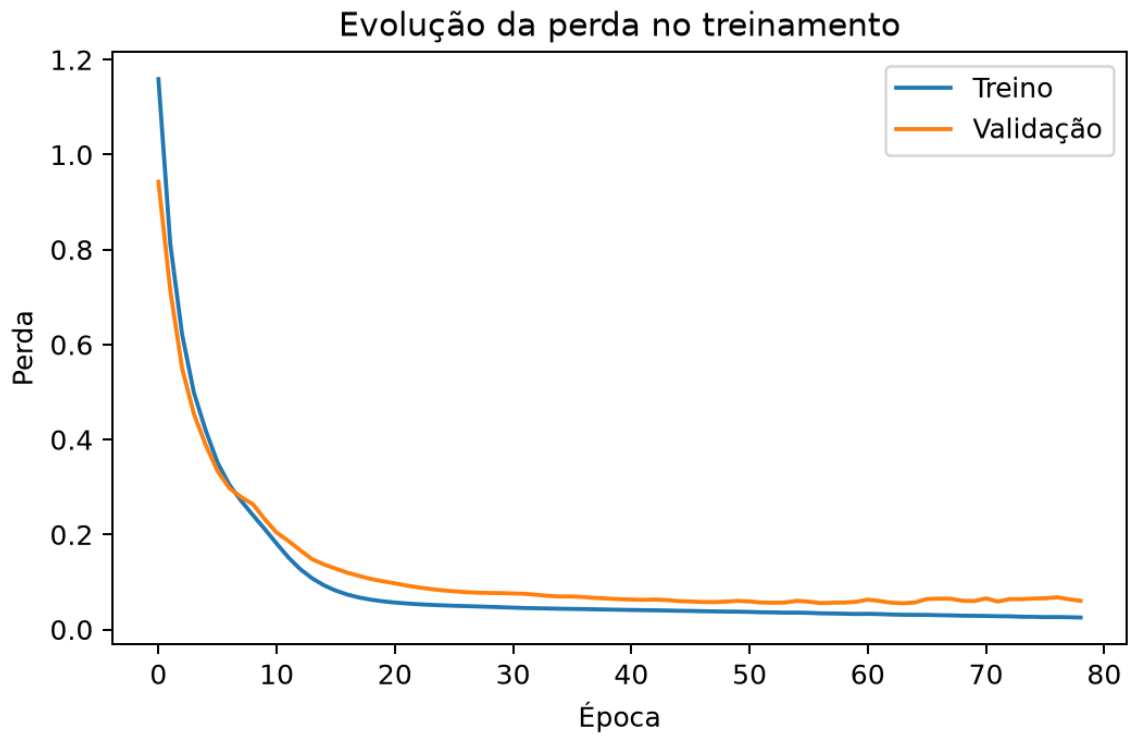
Semente 42 favorece a reprodução. A execução determinística foi habilitada, mas versões de bibliotecas e plataformas diferentes ainda podem apresentar diferenças nos resultados. O conjunto de teste é utilizado apenas após o treinamento.

Indicador	Resultado real
Treino / validação / teste	96 / 24 / 30
Épocas executadas	79
Acurácia de teste	96.67%
Perda de teste	0.1020

Avaliação e previsões

Real / prevista	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	0	10

A matriz tem classes reais nas linhas e previstas nas colunas. A diagonal representa acertos. O teste contém somente 30 flores; a acurácia obtida é uma medida dessa divisão específica e não garante desempenho equivalente em qualquer população ou conjunto de dados.



Medidas sépala e pétala em cm	Previsão
[5.1, 3.5, 1.4, 0.2]	setosa
[6.0, 2.9, 4.5, 1.5]	versicolor
[6.5, 3.0, 5.8, 2.2]	virginica

As três novas entradas são demonstrações e não possuem rótulos coletados independentemente para validação. As probabilidades softmax são saídas do modelo, não garantias de confiança calibrada.

Avaliação dos resultados e verificações

Por que pré-processar e separar treino e teste?

O escalonamento coloca as características em escalas comparáveis, favorecendo a otimização da rede e evitando que diferenças de magnitude dominem o aprendizado. A divisão entre treino e teste permite avaliar generalização em exemplos não usados no ajuste dos pesos. A validação orienta a parada antecipada sem consultar o teste. Ajustar o scaler antes da divisão ou escolher repetidamente o modelo pelo teste pode provocar vazamento e uma avaliação otimista.

Foram conferidos o tamanho do teste, a soma dos elementos da matriz de confusão, a existência de probabilidades finitas e sua soma próxima de um por amostra. Também se confirmou média próxima de zero no treino padronizado. O modelo foi salvo em modelo_iris.keras. Para futuras inferências fora do notebook, é necessário conservar também os parâmetros do scaler e a ordem das características.

O treinamento realizou aprendizagem supervisionada: os exemplos de treino incluem medidas e classes conhecidas. O modelo aprende uma relação entre esses dados e utiliza essa relação para classificar medidas apresentadas posteriormente. O exercício não pretende substituir identificação botânica especializada.

Referências

Roteiro U4 A4, material fornecido. Documento fornecido para a atividade, s.d.

TensorFlow, classificação básica. Disponível em: https://www.tensorflow.org/tutorials/keras/classification. Acesso em: 13 out. 2026.

Scikit-learn, StandardScaler. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Acesso em: 13 out. 2026.

Scikit-learn, Iris. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html. Acesso em: 13 out. 2026.

Codificação completa

Arquivo iris_modelo.py. Inclui os comentários, as verificações e a gravação dos resultados.

```
# %% Bibliotecas e carregamento do Iris
import json
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

tf.keras.utils.set_random_seed(42)
tf.config.experimental.enable_op_determinism()
tf.config.threading.set_inter_op_parallelism_threads(1)
tf.config.threading.set_intra_op_parallelism_threads(1)
iris = load_iris()
X, y = iris.data, iris.target
df = pd.DataFrame(X, columns=iris.feature_names)
df['especie'] = [iris.target_names[i] for i in y]
print('DADOS IRIS:', X.shape, '| classes:', iris.target_names.tolist())
print(df.head().to_string(index=False))

# %% Divisão estratificada e padronização sem vazamento de dados
X_treino_total, X_teste, y_treino_total, y_teste = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
X_treino, X_validacao, y_treino, y_validacao = train_test_split(
    X_treino_total, y_treino_total, test_size=0.20, random_state=42,
    stratify=y_treino_total)
scaler = StandardScaler()
X_treino_pad = scaler.fit_transform(X_treino).astype('float32')
X_validacao_pad = scaler.transform(X_validacao).astype('float32')
X_teste_pad = scaler.transform(X_teste).astype('float32')
print(f'Treino: {len(X_treino)}; validação: {len(X_validacao)}; teste: {len(X_teste)}')

# %% Construção e treinamento da rede neural TensorFlow
modelo = tf.keras.Sequential([
    tf.keras.Input(shape=(4,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax'),
])
modelo.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
               loss='sparse_categorical_crossentropy', metrics=['accuracy'])
modelo.summary()
parada = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=15, restore_best_weights=True)
historico = modelo.fit(
    X_treino_pad, y_treino, validation_data=(X_validacao_pad, y_validacao),
    epochs=150, batch_size=16, verbose=0, callbacks=[parada])
print('Épocas executadas:', len(historico.history['loss']))

# %% Avaliação única no conjunto de teste reservado
perda, acuracia = modelo.evaluate(X_teste_pad, y_teste, verbose=0)
probabilidades = modelo.predict(X_teste_pad, verbose=0)
```

Codificação completa continuação

```
predicoes = probabilidades.argmax(axis=1)
matriz = confusion_matrix(y_teste, predicoes)
print(f'Perda no teste: {perda:.4f}; acurácia: {acuracia:.2%}')
print('Matriz de confusão (linhas reais, colunas previstas):\n', matriz)
print(classification_report(y_teste, predicoes,
      target_names=iris.target_names, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(historico.history['loss'], label='Treino')
ax.plot(historico.history['val_loss'], label='Validação')
ax.set(title='Evolução da perda no treinamento', xlabel='Época', ylabel='Perda')
ax.legend()
fig.tight_layout()
fig.savefig('treinamento_iris.png', dpi=180)
plt.show()

# %% Previsões de três novas entradas demonstrativas
novas_flores = np.array([[5.1, 3.5, 1.4, 0.2],
                         [6.0, 2.9, 4.5, 1.5],
                         [6.5, 3.0, 5.8, 2.2]])
probs = modelo.predict(scaler.transform(novas_flores).astype('float32'), verbose=0)
previsoes_novas = []
for medidas, p in zip(novas_flores, probs):
    especie = str(iris.target_names[int(p.argmax())])
    previsoes_novas.append(especie)
    print(f'Medidas {medidas.tolist()} -> {especie}; probabilidades {p.round(4).tolist()}')
assert len(y_teste) == 30 and matriz.sum() == 30
assert np.isfinite(probabilidades).all()
assert np.allclose(probabilidades.sum(axis=1), 1, atol=1e-5)
assert np.allclose(X_treino_pad.mean(axis=0), 0, atol=1e-5)
print('TESTES: divisão, padronização e probabilidades conferidas.')
modelo.save('modelo_iris.keras')
with open('resultados.json', 'w', encoding='utf-8') as arquivo:
    json.dump({'treino': len(y_treino), 'validacao': len(y_validacao), 'teste': len(y_teste),
               'epocas': len(historico.history['loss']), 'perda': float(perda),
               'acuracia': float(acuracia), 'matriz': matriz.tolist(),
               'previsoes': previsoes_novas}, arquivo, indent=2, ensure_ascii=False)
```

Registro de execução

Captura do notebook com código e saída efetivamente executados no ambiente local. As demais células e saídas estão preservadas no arquivo .ipynb.

Épocas executadas: 79

Avaliação única no conjunto de teste reservado

In [4]:
```
perda, acuracia = modelo.evaluate(X_teste_pad, y_teste, verbose=0)
probabilidades = modelo.predict(X_teste_pad, verbose=0)
predicoes = probabilidades.argmax(axis=1)
matriz = confusion_matrix(y_teste, predicoes)
print(f'Perda no teste: {perda:.4f}; acurácia: {acuracia:.2%}')
print('Matriz de confusão (linhas reais, colunas previstas):\n', matriz)
print(classification_report(y_teste, predicoes,
      target_names=iris.target_names, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(historico.history['loss'], label='Treino')
ax.plot(historico.history['val_loss'], label='Validação')
ax.set(title='Evolução da perda no treinamento', xlabel='Época', ylabel='Perda')
ax.legend()
fig.tight_layout()
fig.savefig('treinamento_iris.png', dpi=180)
plt.show()
```

```
Perda no teste: 0.1020; acurácia: 96.67%
Matriz de confusão (linhas reais, colunas previstas):
 [[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

